CAK3EAB3 KOMPUTASI AWAN DAN TERDISTRIBUSI

Tugas-5

KOORDINASI PADA SISTEM TERDISTIBUSI

Kelompok: xxx

KELAS IF-XX-XX

Dosen: XXX



PROGRAM STUDI XXX

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM

BANDUNG

2026

Ketentuan:

1. Dikerjakan secara kelompok.
2. Dikerjakan langsung pada lembar soal.
3. Jawaban ditulis dengan tulisan tangan yang rapi.
4. Tulisan tangan harus terbaca dengan baik. Tulisan yang tidak terbaca diberi nilai 0
5. Untuk soal perhitungan, maka cara atau langkah pengerjaan **wajib** dituliskan.
6. Setiap anggota kelompok wajib mengerjakan soal (ikut berkontribusi).
7. Anggota kelompok yang tidak ikut mengerjakan maka nilainya nol, nama mahasiswa tersebut tidak perlu ditulis di tabel.
8. Unggah jawaban ke LMS diwakili oleh salah satu anggota kelompok. Lembar fisik dikumpulkan pada saat pertemuan di kelas.
9. Lengkapi tabel pernyataan di bawah ini sebagai syarat nilai diinput.

PERNYATAAN

Saya menyatakan bahwa saya:

(a) benar-benar ikut mengerjakan soal

(b) telah menjelaskan jawaban saya ke teman anggota kelompok

(c) telah memahami penjelasan dari teman anggota kelompok

(d) tidak mencontek jawaban dari kelompok lain

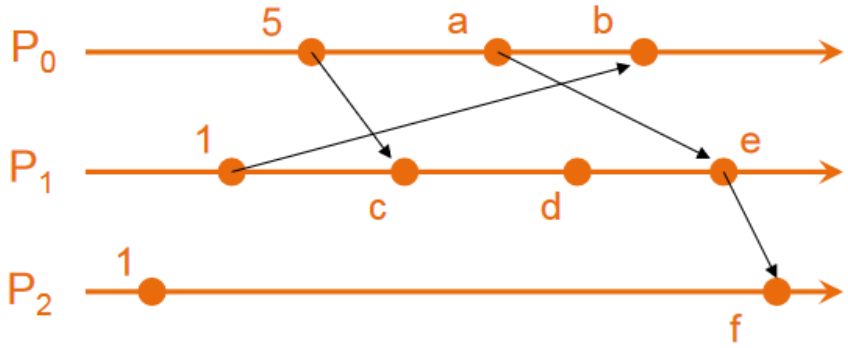
(e) tidak memberikan jawaban PR ini ke kelompok lain

Nama	NIM	Kontribusi Tugas	Tanda Tangan

TUGAS 5

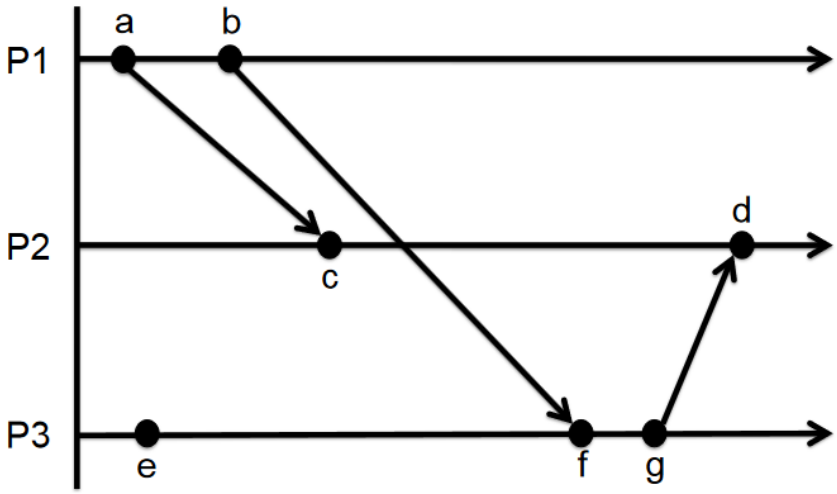
1. **Logical Clock**
   Jawablah pertanyaan-pertanyaan berikut ini:

   a. Tentukan Lamport timestamp pada diagram berikut ini



Jawaban:
Aturan Lamport: setiap event lokal/kirim, LC = LC + 1; saat menerima pesan dengan timestamp t, LC = max(LC, t) + 1.
P0: event awal = 5 (kirim ke c); a = 5 + 1 = 6 (kirim ke e); b = terima dari event P1 (timestamp 1) → max(6, 1) + 1 = 7.
P1: event awal = 1 (kirim ke b); c = terima dari P0 (timestamp 5) → max(1, 5) + 1 = 6; d = 6 + 1 = 7; e = terima dari a (timestamp 6) → max(7, 6) + 1 = 8 (kirim ke f).
P2: event awal = 1; f = terima dari e (timestamp 8) → max(1, 8) + 1 = 9.
Hasil: a = 6, b = 7, c = 6, d = 7, e = 8, f = 9.

   b. Gunakan vector clock untuk menentukan timestamp semua event



Jawaban:
Aturan vector clock (urutan vektor [P1, P2, P3], semua dimulai dari [0,0,0]): (1) sebelum setiap event, proses menaikkan komponennya sendiri; (2) saat

mengirim, vektor ikut dikirim; (3) saat menerima, VC = max(VC lokal, VC pesan) per komponen, lalu komponen sendiri dinaikkan 1.
P1: a = [1,0,0] (kirim ke c); b = [2,0,0] (kirim ke f).
P2: c = max([0,0,0],[1,0,0]) lalu P2+1 = [1,1,0]; d = max([1,1,0],[2,0,3]) lalu P2+1 = [2,2,3].
P3: e = [0,0,1]; f = max([0,0,1],[2,0,0]) lalu P3+1 = [2,0,2]; g = [2,0,3] (kirim ke d).
Ringkasan: a = [1,0,0], b = [2,0,0], c = [1,1,0], d = [2,2,3], e = [0,0,1], f = [2,0,2], g = [2,0,3].

c. Menggunkan gambar b, tentukan apakah event c dan event g terjadi secara **konkuren atau happens before** jika menggunakan vector clock?
Jawaban:
Dari jawaban 1.b: VC(c) = [1,1,0] dan VC(g) = [2,0,3].
Syarat happens-before: c → g jika VC(c) ≤ VC(g) untuk semua komponen dan minimal satu komponen lebih kecil.
Bandingkan per komponen: P1: 1 < 2, P2: 1 > 0, P3: 0 < 3.
Karena komponen P2 pada c lebih besar (1 > 0) sementara komponen P1 dan P3 lebih kecil, maka VC(c) ⋦ VC(g) dan VC(g) ⋦ VC(c).
Kesimpulan: event c dan event g terjadi secara KONKUREN (c ‖ g), tidak ada hubungan happens-before di antara keduanya. Hal ini juga terlihat dari diagram: tidak ada rantai pesan dari c ke g maupun dari g ke c (pesan dari g baru diterima P2 pada event d, setelah c).

2. **Reticulum Announce**
   a. Jelaskan apa yang dimaksud dengan announce pada reticulum!
   Jawaban:
   Announce adalah paket khusus (tipe ANNOUNCE) yang dipancarkan oleh sebuah destinasi untuk memberitahukan keberadaannya ke jaringan. Karena Reticulum tidak memiliki alamat terpusat (coordination-less), announce menjadi satu-satunya cara node lain mengetahui bahwa suatu destinasi ada, dapat dijangkau, dan bagaimana mengenkripsi data untuknya.
   Isi announce: public key identitas (X25519 untuk enkripsi + Ed25519 untuk tanda tangan), name hash, random hash (berisi timestamp), ratchet key (opsional), tanda tangan digital, dan app_data opsional. Dengan menerima announce, node lain memperoleh public key tujuan sekaligus membangun jalur (path) menuju destinasi tersebut.
   Potongan source code pendukung (RNS/Destination.py):

```
A destination can also announce its presence on the network, which will
distribute
```

```
necessary keys for encrypted communication with it.   # docstring class
Destination
def announce(self, app_data=None, path_response=False, attached_interface=None,
tag=None, send=True):
    """Creates an announce packet for this destination and broadcasts it on all
    relevant interfaces. Application specific data can be added to the
announce."""
announce_data =
self.identity.get_public_key()+self.name_hash+random_hash+ratchet+signature
if app_data != None: announce_data += app_data
```

b. Jelaskan cara kerja mekanisme announce pada reticulum!

Jawaban:

1) Pembuatan: aplikasi memanggil destination.announce(). Hanya destinasi bertipe SINGLE dengan arah IN yang boleh announce. Reticulum membuat random_hash (5 byte acak + 5 byte timestamp), lalu menandatangani gabungan hash destinasi + public key + name hash + random hash (+ ratchet + app_data) dengan kunci privat Ed25519.

2) Pengiriman: paket bertipe ANNOUNCE dikirim (broadcast) ke semua interface yang relevan.

3) Validasi: node penerima memeriksa tanda tangan dan mencocokkan hash destinasi dengan hash dari name_hash + identity hash. Announce yang tidak valid dibuang; announce duplikat dikenali lewat random_hash.

4) Propagasi: Transport Node menyimpan announce di announce table, menaikkan hop count, lalu me-rebroadcast setelah jeda acak (PATHFINDER_RW) dengan sejumlah retry (PATHFINDER_R). Announce tidak diteruskan lagi jika sudah melewati batas PATHFINDER_M = 128 hop. Penggunaan bandwidth untuk announce dibatasi (announce cap, default 2% per interface) agar jaringan tidak banjir.

5) Penerimaan di aplikasi: node lain menyimpan public key (Identity.remember) dan path ke destinasi, serta memanggil announce handler yang didaftarkan lewat RNS.Transport.register_announce_handler().

Potongan source code pendukung (RNS/Destination.py, method announce):

```
if self.type != Destination.SINGLE:
    raise TypeError("Only SINGLE destination types can be announced")
random_hash = RNS.Identity.get_random_hash()[0:5]+int(time.time()).to_bytes(5,
"big")
signed_data =
self.hash+self.identity.get_public_key()+self.name_hash+random_hash+ratchet
if app_data != None: signed_data += app_data
signature = self.identity.sign(signed_data)
announce_packet = RNS.Packet(self, announce_data, RNS.Packet.ANNOUNCE, context
= announce_context,
    attached_interface = attached_interface, context_flag=context_flag)
if send: announce_packet.send()
```

RNS/Transport.py (konstanta dan pemrosesan announce):

```
PATHFINDER_M  = 128   # Max hops
PATHFINDER_R  = 1     # Retransmit retries
PATHFINDER_RW = 0.5   # Random window for announce rebroadcast
if packet.packet_type == RNS.Packet.ANNOUNCE:
    if RNS.Identity.validate_announce(packet): ...
        Transport.announce_table[packet.destination_hash] = [now,
retransmit_timeout, retries,
            received_from, announce_hops, packet, local_rebroadcasts,
block_rebroadcasts, attached_interface]
```

c. Bagaimana routing dibuat pada reticulum?

Jawaban:

Reticulum tidak memakai alamat IP atau tabel routing yang dikonfigurasi manual. Routing dibentuk otomatis dari announce: setiap Transport Node yang menerima announce valid mencatat entri path table berisi hash tujuan, next hop (node yang meneruskan announce, received_from), jumlah hop, waktu kedaluwarsa, dan interface penerima. Bila satu tujuan diterima lewat beberapa jalur, jalur dengan hop paling sedikit (atau yang lebih baru bila jalur lama kedaluwarsa) yang disimpan.

Saat mengirim paket ke tujuan yang hop-nya > 1, Transport menyisipkan header tipe HEADER_2 berisi transport_id = hash next hop, sehingga setiap node hanya perlu tahu "lompatan berikutnya" (hop-by-hop routing), bukan rute lengkap. Jika path belum diketahui, node mengirim path request ke destinasi khusus rnstransport.path.request, dan node yang tahu jalurnya (atau tujuan itu sendiri) membalas dengan announce bertipe PATH_RESPONSE. Path kedaluwarsa setelah PATHFINDER_E (default 1 minggu) dan diperbarui oleh announce baru.

Potongan source code pendukung (RNS/Transport.py):

```
PATHFINDER_E = 60*60*24*7   # Path expiration of one week
# di Transport.inbound(), setelah announce tervalidasi:
Transport.path_table[packet.destination_hash] = [now, received_from,
    announce_hops, expires, random_blobs, packet.receiving_interface,
packet.packet_hash]
# di Transport.outbound(), saat mengirim ke tujuan multi-hop:
if Transport.path_table[packet.destination_hash][IDX_PT_HOPS] > 1:
    new_flags = (RNS.Packet.HEADER_2) << 6 | (Transport.TRANSPORT) << 4 |
(packet.flags & 0b00001111)
    new_raw = struct.pack("!B", new_flags) + packet.raw[1:2]
        + Transport.path_table[packet.destination_hash][IDX_PT_NEXT_HOP] +
packet.raw[2:]
# Permintaan jalur jika path belum diketahui:
def request_path(destination_hash, on_interface=None, tag=None,
recursive=False):
    path_request_dst = RNS.Destination(None, RNS.Destination.OUT,
RNS.Destination.PLAIN,
        Transport.APP_NAME, "path", "request")
```

Catatan: nama tabel/indeks dapat sedikit berbeda antarversi RNS (versi lama memakai nama destination_table); cocokkan dengan versi source code yang Anda gunakan.

Untuk 2.a, 2.b. 2.c: tunjukkan baris-baris pada source code reticulum untuk mendukung jawaban Anda

d. Buatlah program menggunakan reticulum untuk mengirim pesan dari node A. Lampirkan source code program dan screenshot hasilnya.

Jawaban:

Program terdiri dari dua bagian: node_b_penerima.py (destinasi IN yang melakukan announce dan menampilkan pesan masuk) dan node_a_pengirim.py (Node A yang mencari path ke Node B lalu mengirim pesan). Instalasi: pip install rns. Jalankan Node B terlebih dahulu, salin hash tujuan yang tampil, lalu jalankan Node A dengan hash tersebut. Kedua node dapat dijalankan pada dua komputer dalam satu LAN (AutoInterface aktif secara default di ~/.reticulum/config) atau pada satu komputer.

Source code node_b_penerima.py:

```
import RNS, time
APP_NAME = "tugas5"

def on_packet(data, packet):
    RNS.log("Pesan diterima: " + data.decode("utf-8"))
```

```
reticulum = RNS.Reticulum()
identity  = RNS.Identity()
dest = RNS.Destination(identity, RNS.Destination.IN,
                       RNS.Destination.SINGLE, APP_NAME, "pesan")
dest.set_packet_callback(on_packet)
dest.set_proof_strategy(RNS.Destination.PROVE_ALL)
RNS.log("Node B siap, hash tujuan: " + RNS.prettyhexrep(dest.hash))
while True:
    dest.announce()
    RNS.log("Announce dikirim")
    time.sleep(30)
```

Source code node_a_pengirim.py:

```
import RNS, sys, time
APP_NAME = "tugas5"

reticulum = RNS.Reticulum()
dest_hash = bytes.fromhex(sys.argv[1])   # hash tujuan dari Node B
pesan     = sys.argv[2] if len(sys.argv) > 2 else "Halo dari Node A!"

if not RNS.Transport.has_path(dest_hash):
    RNS.log("Path belum ada, mengirim path request...")
    RNS.Transport.request_path(dest_hash)
    while not RNS.Transport.has_path(dest_hash):
        time.sleep(0.1)

identity_b = RNS.Identity.recall(dest_hash)
dest_b = RNS.Destination(identity_b, RNS.Destination.OUT,
                         RNS.Destination.SINGLE, APP_NAME, "pesan")
packet  = RNS.Packet(dest_b, pesan.encode("utf-8"))
receipt = packet.send()
RNS.log("Pesan dikirim ke " + RNS.prettyhexrep(dest_hash))

def delivered(r):
    RNS.log("Pesan terkonfirmasi diterima Node B (RTT %.3f s)" % r.get_rtt())
receipt.set_delivery_callback(delivered)
time.sleep(5)
```

Cara menjalankan:

```
python node_b_penerima.py
python node_a_pengirim.py <hash_tujuan_node_B> "Halo dari Node A!"
```

Hasil eksekusi program (simulasi tampilan terminal):

Contoh output (simulasi) — Terminal 1: Node B (penerima)

```
user@node-b:~/tugas5$ python node_b_penerima.py
[2026-10-19 10:15:02] [Notice] Node B siap, hash tujuan: <a3f91c0e7b2d4568e1f0c9a7b3d25e14>
[2026-10-19 10:15:02] [Notice] Announce dikirim
[2026-10-19 10:15:20] [Notice] Pesan diterima: Halo dari Node A!
[2026-10-19 10:15:32] [Notice] Announce dikirim
^C
```

Contoh output (simulasi) — Terminal 2: Node A (pengirim)

```
user@node-a:~/tugas5$ python node_a_pengirim.py \
    a3f91c0e7b2d4568e1f0c9a7b3d25e14 "Halo dari Node A!"
[2026-10-19 10:15:19] [Notice] Path belum ada, mengirim path request...
[2026-10-19 10:15:20] [Notice] Pesan dikirim ke <a3f91c0e7b2d4568e1f0c9a7b3d25e14>
[2026-10-19 10:15:20] [Notice] Pesan terkonfirmasi diterima Node B (RTT 0.042 s)
user@node-a:~/tugas5$
```